Bicol University
College of Science
IT Department
Legazpi City, Albay

Artistryx: REST API Enhancement and Multi-Client Implementation Documentation

I. Overview

This REST API is designed to handle user authentication and seller shop management, returning all responses in JSON format. It supports key features such as user registration, login validation, and create, read, and update operations for seller shops, while implementing multi-status responses using proper HTTP status code. It is accessible through multiple clients including HTML/JavaScript and Python.

II. API Request and Response Summary

Feature	Method	Endpoint	Parameters	Sample Request	Sample Response
Register	POST	http://localhost/[your-folder]/service.php	user email password	{"action": "register", "username": "Nameless", "email": "nameless@gmail.com", "password": "password1234" }	{ "status": "success", "code": 201, "message": "Registration successful.", "user": { "id": 5, "username": "Nameless", "email": "nameless@gmail.com" }}
Login	POST	http://localhost/[your-folder]/service.php	email password	{"action": "login", "email": "nameless@gmail.com", "password": "password1234"}	{ "status": "success", "code": 200, "message": "Login successful.", "user": { "id": 5, "username": "Nameless", "email": "nameless@gmail.com" }}
Register Shop	POST	http://localhost/[your-folder]/service.php	userId shopName	{"userId": 5, "shopName": "'Pangit Peter"}	{"status": "success", "code": 201, "message": "Seller shop 'Pangit Peter' created successfully.", "details": { "sellerId": 3, "userId": 5, "shopName": "Pangit Peter", "shopStatus": "Pending"}}
View Shop	GET	http://localhost/[your-folder]/service.php	userId	N/A	{"status": "has_shop", "code": 200, "message": "You already have a registered shop.", "hasShop": true, "details": { "sellerId": 3,

					"userId": 5, "username": "Nameless", "shopName": "Pangit Peter", "shopStatus": "Pending"}}
Update Shop	PUT	http://localhost/[your-folder]/service.php	userId newShopName	{"userId": 5, "newShopName": "Fun Comics"}	{"status": "success", "code": 200, "message": "Shop name updated successfully.", "details": { "sellerId": 3, "userId": 5, "oldShopName": "Pangit Peter", "newShopName": "Fun Comics" }}

III. Error Handling

The API returns appropriate HTTP status codes and JSON error messages for every failure case to ensure proper error handling. These codes include 400 if required parameters are missing or input fails validation, 401 if the entered password does not match the stored hashed password of the account, 404 if the email does not exist in the database or if no stored shop name is found for a given account during update, 409 if the entered username, email, or shop name is already taken by another user or there is an attempt to register a second shop, 405 if a method other than GET, POST, or PUT is used to call the endpoint, and 500 if database fails to connect.

IV. Interaction Flow

The flow follows a client-server interaction. It begins with user registration, which involves submitting the required parameters, such as username, email, and password. Then, users can login to their accounts by entering their email and password. These actions are enabled by the client sending requests to the API, which validates the input and interacts with the database. After logging in, the user can become a seller by creating a shop, which they can also view and update. These actions are performed through sending corresponding HTTP requests, such as POST, GET, and PUT, to the server. Then, the server processes these requests from the client, performs validations, and returns responses.

V. Technology Used

Several technologies were utilized in developing and implementing REST API enhancements and multi-client implementations for Artistryx. Among these is the PHP programming language, which manages all incoming HTTP requests, processes system logics, validates user inputs, and returns JSON responses. For the database, MySQL is used to store records of all user accounts and sellers' shops. This artistryx_db database specifically contains "users" and "sellers" tables. Furthermore, Apache (XAMPP) is used as a local web server environment to host service.php. It serves a localhost and enables clients to send requests to API during development and testing phases. For the client-side, Python is used to develop the first client, client1.py. It provides a CLI app that communicates with API using requests library to perform operations like registration, login, and shop handling. Lastly, the HTML/JavaScript is used to develop the second client, client1.html. It allows users to interact with functionalities through a web form.